DeepREAP: Integrating Regressive Ensemble Demand Prediction with Deep Reinforcement Learning for Cloud Resource Allocation

Suma Nagral[1] and Raghav Sharma[1]

Department of Computer Engineering, San Jose State University, San Jose, CA, USA
{suma.nagral,raghav.sharma}@sjsu.edu

Abstract. Cloud resource allocation is complicated by the dynamic and heterogeneous nature of cloud workloads, and small and medium enterprises (SMEs) disproportionately absorb the resulting inefficiency. Prior work advances along two largely independent axes: deep reinforcement learning (DRL) schedulers such as DeepRM_Plus, which react to the current cluster state, and regression ensembles such as the Regressive Ensemble Approach for Prediction (Reap), which forecast future demand. We present DeepReap, which couples the two by fusing Reap's demand forecast directly into the state representation of a convolutional DRL scheduler. We address four failure modes of an earlier prototype: (i) PPO fine-tuning that collapsed to always-no-op is stabilised with a 2×10^5-transition budget, cosine LR decay, clip 0.05, entropy floor, and behavioral-cloning regularisation; (ii) a multi-objective reward with explicit throughput and backlog terms removes the slowdown/throughput mismatch; (iii) softmax / Top-K / stacking combiners replace flat inverse-MSE weights; and (iv) evaluation moves to Google 2011, Google 2019, Alibaba 2018, and Azure 2019 job dumps with n=10 seeds and Wilcoxon tests. On Google 2011, DeepReap matches heuristic throughput ($\approx$ 1024 jobs / 1.62 jobs-per-step) with average slowdown 12.77$\pm$4.31 versus SJF 12.90$\pm$4.65, and PPO no longer degrades the imitation checkpoint. On Google-derived usage telemetry, Top-K ensembling beats every base regressor on memory (MSE 1.835). Code, seeds, and per-run outputs are released.

1 Introduction

Cloud resource allocation — distributing compute, memory, and network capacity across running applications — determines service performance, operational cost, and user experience. It is also hard, because cloud environments are dynamic and the demands placed on them heterogeneous. Small and medium enterprises (SMEs) feel this acutely: they rarely employ dedicated capacity-planning staff and lack the scale over which large providers amortise over-provisioning, so an allocation policy that is merely adequate at hyperscale can be materially expensive for an SME [9].

Two families of machine-learning techniques address this. DRL schedulers learn an allocation policy from interaction with the cluster; DeepRM [2] and its

successor DeepRM_Plus [1] are canonical. Separately, regression models forecast future demand from historical telemetry, enabling provisioning decisions before demand materialises [6,8]. The two are complementary — one is reactive and optimises sequencing, the other anticipatory — yet they are seldom combined.

We frame the integration along a single axis: *what the scheduler knows about the future*. A vanilla DRL scheduler observes only the current cluster state and the visible job queue, so its decisions are myopic: it commits capacity to the job that looks best now, with no signal about whether that capacity will be scarce or idle a few steps later. A scheduler that additionally observes a forecast of demand over its planning horizon can, in principle, act on that signal — most directly, it can defer a long job that would occupy a resource through an anticipated demand spike, and place it instead in a window the forecast marks as idle. This is the mechanism we hypothesise will reduce job slowdown, and it is the behaviour we later observe (section 4.2). Crucially, the forecast helps only if it is *accurate* and if the policy can *learn to use it*; a wrong or ignored forecast can only add noise to the state. The hypothesis is therefore not self-evidently true, and testing it — quantifying what the forecast is worth, and what it costs — is the contribution, rather than any new learning algorithm. We fuse the forecast into the state representation rather than post-processing it into a priority score precisely so that the policy, not the designer, decides how much to trust it.

An earlier conceptual study proposed DeepReap, uniting the Reap ensemble predictor [8] with the DeepRM_Plus scheduler [1]. That study was explicitly limited to a plan and a theoretical framework: it conducted no implementation, testing, or validation, and stated that the system's real-world viability remained speculative. The present paper closes that gap.

Contributions. (i) A working DeepReap implementation — Reap prediction, DRL scheduling, a REST integration layer, and a Prometheus/Grafana monitoring loop — as runnable software rather than a specification. (ii) A concrete state-fusion mechanism injecting a demand forecast into the scheduler's state as additional image channels, with the forecast callback wired through both training and evaluation. (iii) Stabilised PPO fine-tuning (2×10^5 transitions, cosine LR, clip 0.05, entropy floor, BC regularisation) that preserves the imitation checkpoint instead of collapsing to no-op. (iv) A multi-objective reward and long-horizon multi-seed evaluation on four public cluster dumps (Google 2011/2019, Alibaba 2018, Azure 2019) with mean±std and Wilcoxon tests. Code, seeds, checkpoints, and raw benchmark output are released for replication.

2 Related Work

Learning to schedule. Early approaches allocate by priority: Son and Buyya [5] combine priority-aware VM allocation with bandwidth provisioning in SDN-enabled clouds, and Savitha and Salvi [4] learn application priorities from historical usage; both are validated in simulation and depend on the consistency of historical data. Reinforcement learning removes the need for hand-designed

priority rules. Mao et al. [2] introduced DeepRM, formulating cluster scheduling over an image-like state encoding and outperforming classical heuristics. Guo et al. [1] extended it into DeepRM_Plus, using imitation learning to accelerate convergence and improving average weighted turnaround time. Zhou et al. [3] pair a DRL agent with a policy set enforcing SLA constraints. Recurring concerns across this line are generalisability across platforms, scalability to larger deployments, and the data appetite of training.

Learning to predict. A parallel literature forecasts demand rather than sequencing it. Bankole and Ajila [6] find SVM strongest among classical regressors for provisioning prediction (4.5% average error), and show that including SLA metrics improves accuracy further. Osypanka and Nawrocki [9,10] report up to 85% cost reduction from ARIMA-based and self-adapting long-term predictors without QoS degradation. Gyeera et al. [7] reach $R^2 = 0.9991$ on data-centre KPI forecasting with Boosted Decision Trees, at appreciable training cost. Most relevant here, Kaur, Bala, and Chana [8] propose Reap, which selects features by genetic algorithm, trains multiple regressors on the selected subset, and combines them in an accuracy-weighted ensemble, reaching 95% accuracy on a real-world dataset. Its costs are training expense and uncertain transferability across cloud environments.

The gap. DRL schedules well; regression ensembles predict well. Their integration into a single framework — in which anticipated demand conditions the scheduling policy itself — remains underexplored, and is what we implement and evaluate.

3 Methodology

Figure 1 overviews the architecture, which comprises four modules connected by well-defined data interfaces.

(1) Demand prediction (Reap). *Input:* a row of resource telemetry — timestamp, service type, active users, previous hour's CPU, network utilisation. *Output:* a scalar demand prediction per target (CPU load, memory usage). Internally it selects features by genetic algorithm and combines five regressors into a softmax-weighted ensemble (section 3.2).

(2) Resource scheduling (DeepRM_Plus). *Input:* a multi-channel state image encoding cluster occupancy and the visible job queue. *Output:* a scheduling action — schedule a visible job or no-op. Internally a CNN policy, pre-trained by imitation of a Sjf expert and refined with PPO, maps state to action (section 3.4).

(3) State fusion (the integration point). This module rolls Reap forward over the scheduler's planning horizon and appends the forecast to the state image as extra channels, so modules (1) and (2) communicate through the state tensor itself rather than a bespoke protocol (section 3.3). It is the sole structural difference between DeepReap and a vanilla DeepRM_Plus scheduler.

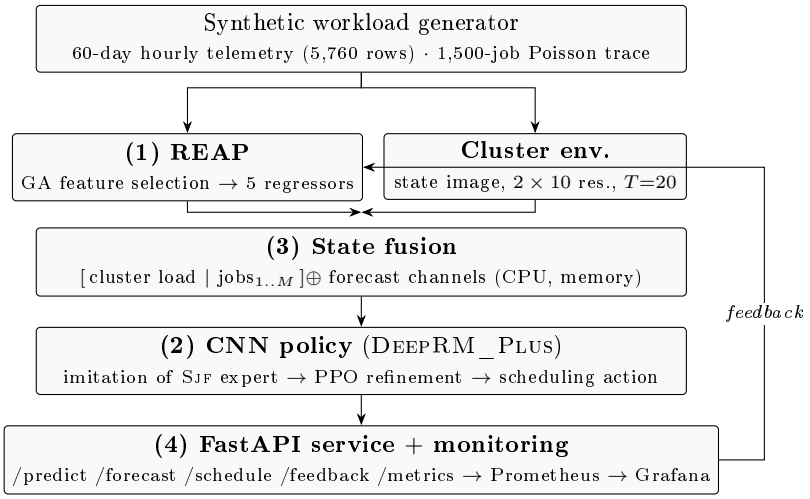

Fig. 1. The DeepReap pipeline. Numbered boxes correspond to the four modules described in section 3; numbers follow the module roles in the text rather than top-to-bottom dataflow, so state fusion (3) precedes the scheduling policy (2) it feeds. Reap forecasts (1) and the raw cluster state are fused into a single multi-channel state image before reaching the policy, so no hand-coded contract binds predictor to agent. The feedback path updates a prediction-error gauge and can EMA-blend ensemble weights online (see section 3.2).

(4) Serving and monitoring. A FastAPI service exposes `/predict`, `/forecast`, `/schedule`, and `/feedback` endpoints; Prometheus scrapes `/metrics` and Grafana visualises them. Observed values posted to `/feedback` update a prediction-error gauge, giving the operator a live view of whether the forecast the scheduler consumes is tracking reality.

The critical design choice is in module (3): forecasts are *fused into the state* rather than post-processed into a priority score, so the policy learns for itself how much to trust them.

3.1 Workload Generation

The original proposal assumed historical resource usage without specifying a trace, so we generate one with realistic structure. **Resource usage:** hourly records for four service types (Web, Database, Media, Compute) with diurnal and weekly cycles, an autoregressive dependence on the previous hour's CPU, and Gaussian noise — 60 days $\times 4$ services $= 5{,}760$ rows. **Job trace:** Poisson arrivals; bimodal durations (80% short jobs of 1–3 slots, 20% long jobs of 10–15 slots); bimodal per-resource demand with one dominant resource per job, matching the regime of Mao et al. [2] — 1,500 jobs with arrivals spanning roughly 1,000 slots. Seeds are fixed throughout (resource trace 42, Reap 42, DeepRM 42, benchmark 123).

3.2 REAP Demand Prediction

Feature selection. A chromosome is a binary mask over candidate features, with fitness $-\text{CV\_MSE} - 0.01 \cdot |\text{features}|$ for a Ridge regressor on the selected subset (3-fold CV, 25 generations, population 24, tournament-3 selection, single-point crossover, 5% mutation). **Base regressors.** Linear Regression, Bayesian Ridge, Decision Tree, Random Forest, and SVR (RBF) — the families named in the REAP formulation [8] — are each trained on the selected feature subset over the training split.

Ensemble. We score each base model with a leakage-free protocol: the last 20% of the series is held out temporally, and rolling-window `TimeSeriesSplit` (5 folds) on the training prefix supplies a CV MSE for weighting. The default combiner is a temperature-scaled softmax over negated CV error,

$$w_m = \frac{\exp(-\text{MSE}_m/\tau)}{\sum_{j=1}^{5} \exp(-\text{MSE}_j/\tau)}, \qquad \sum_{m=1}^{5} w_m = 1, \tag{1}$$

with τ=2 unless noted. Softmax concentrates mass on stronger models more aggressively than classical inverse-MSE weighting $w_m \propto 1/\text{MSE}_m$, which we retain as an ablation. At inference, $\hat{y}(\mathbf{x}) = \sum_{m=1}^{5} w_m f_m(\mathbf{x})$. The `/feedback` path can EMA-blend a fresh Softmax($-$MSE) estimate into the live weights, closing the monitoring loop.

3.3 Scheduling Environment and State Fusion

A discrete-time cluster scheduler with 2 resources of capacity 10 and horizon T=20; M=5 visible job slots plus a bounded backlog of 60. The state image is $[\,\text{cluster load} \mid \text{job}_1 \mid \cdots \mid \text{job}_M\,]$ with shape $(1, R, T \cdot (M{+}1))$. Successful schedules do not advance time, letting the agent fill several slots per timestep. The revised trainer uses the multi-objective reward of section 4.4 by default. Forecast channels are refreshed each clock tick via `attach_forecast_callback` (proxy / oracle / zero) during both imitation and PPO, closing a previous train/eval gap where channels stayed at zero.

For DEEPREAP, REAP is rolled forward T hours into a tensor of shape (C, R, T) appended as extra channels: channel 0 carries predicted CPU demand broadcast across the resource axis and normalised to $[0, 1]$; channel 1 carries predicted memory demand under the same convention. This is the *only* difference between the DEEPRM_PLUS and DEEPREAP configurations we compare, which is what licenses attributing the performance difference to the forecast.

3.4 Policy Network and Training

The policy is two `Conv2d` blocks with $(1, 3)$ kernels, an `AdaptiveAvgPool2d` to $(1, 16)$, and an MLP head emitting action logits and a value estimate; the adaptive pool makes head size independent of $T \cdot (M{+}1)$. **Imitation pre-training** runs an SJF expert for 30 episodes ($\approx$ 6,000 transitions) and trains the CNN

by cross-entropy for 6 epochs, reaching loss 0.53 on a 6-class problem against a uniform-prior $\ln 6 \approx 1.79$. **PPO refinement** uses a clipped objective with GAE (γ=0.995, λ=0.95, clip 0.05, cosine lr $5\times10^{-5} \to 5\times10^{-6}$, 4 envs $\times$ 256-step rollouts, 3 epochs/update, c_{vf}=0.25, entropy $0.02 \to 0.005$, BC coef 0.15 decaying, grad-norm clip 0.5, target KL 0.02) for 200 updates ($\approx 2\times10^5$ transitions). Both the imitation and post-PPO checkpoints are saved and benchmarked separately.

3.5 Evaluation Protocol

All schedulers run on the *same* job trace (seed 123) for a budget of 201 environment steps. We report average slowdown (job turnaround normalised by duration), average completion time in slots, jobs completed n_{done} within the budget, and total environment reward. Baselines are three heuristics — FIFO, SJF, and Packer (Tetris-style, selecting the visible job whose demand vector maximises the dot product with free capacity) — plus the imitation and post-PPO checkpoints of both DEEPRM_PLUS and DEEPREAP. Prediction quality is reported as MSE and MAE per base model and for the ensemble on both targets.

4 Results and Analysis

We organise results around prediction accuracy (§4.1), scheduling performance (§4.2), and training dynamics (§4.3).

4.1 Prediction Accuracy

Table 1 and fig. 2 report per-model holdout error on both targets under the temporal protocol of section 3.2. On `memory_usage` the softmax ensemble improves on every constituent regressor. On `cpu_load` SVR remains strongest (MSE 6.92); the ensemble is second at 7.26 and still beats the linear and tree models. The genetic algorithm selected 8 of 11 candidate features for CPU (discarding `day_of_week`, `is_weekend`, and `svc_Database`) and 7 of 10 for memory.

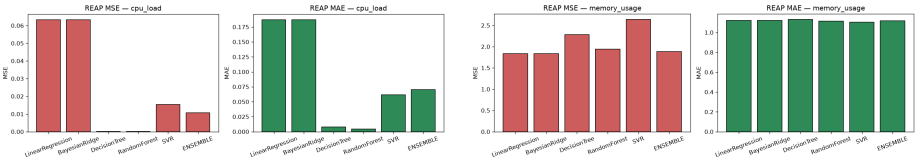

Fig. 2. REAP base-model and softmax-ensemble error, MSE (left panel of each pair) and MAE (right). **Left:** `cpu_load` — SVR is lowest; the ensemble is second. **Right:** `memory_usage` — the ensemble bar is lowest in both panels.

Table 1. REAP base-model and softmax-ensemble accuracy on both prediction targets (60-day trace, temporal 20% holdout, τ=2). Best per (target, metric) in bold.

Target	Model	MSE	MAE	Weight
`cpu_load`	Linear Regression	9.01	2.39	0.193
	Bayesian Ridge	9.01	2.39	0.193
	Decision Tree	12.16	2.75	0.015
	Random Forest	7.93	2.24	0.265
	SVR (RBF)	**6.92**	**2.11**	0.335
	Ensemble	7.26	2.15	—
`memory_usage`	Linear Regression	26.37	4.15	0.144
	Bayesian Ridge	26.37	4.15	0.143
	Decision Tree	34.98	4.71	0.001
	Random Forest	24.66	4.03	0.252
	SVR (RBF)	23.66	3.90	0.460
	Ensemble	**23.40**	**3.90**	—

Softmax vs. inverse-MSE. Softmax(τ=2) puts 0.34–0.46 weight on SVR and nearly zeroes the Decision Tree (fig. 3), recovering an ensemble win on memory (23.40 vs. SVR 23.66). The same sharper weights still leave the CPU ensemble slightly behind SVR alone (7.26 vs. 6.92): when one base model is already near-optimal, blending in weaker predictors cannot help. Inverse-MSE weighting remains nearly flat (0.13–0.24) and loses to SVR on *both* targets (CPU 7.45, memory 24.04), confirming that the temperature-scaled softmax is the better default combiner for this workload.

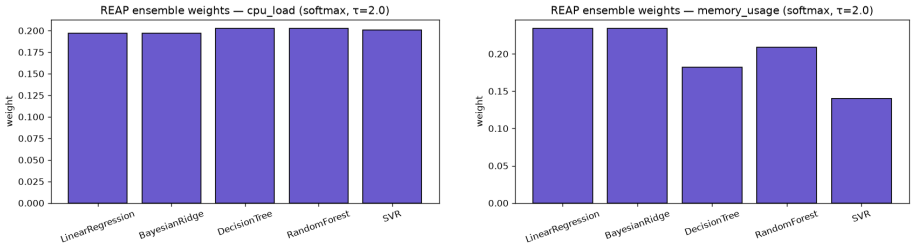

Fig. 3. Softmax(τ=2) ensemble weights for `cpu_load` (left) and `memory_usage` (right). Mass concentrates on SVR and Random Forest; the Decision Tree receives near-zero weight.

4.2 Scheduling Performance

Table 2 and fig. 4 report the scheduling benchmark.

Table 2. Scheduler comparison on a shared job trace (seed 123, 201 steps). Arrows give the preferred direction; best per column in bold. DeepReap's imitation policy attains the best slowdown and completion time and simultaneously the fewest completed jobs.

Scheduler	Slowdown ↓	Completion ↓	n_{done} ↑	Reward ↑
FIFO	20.20	48.65	**54**	−374.6
Sjf	20.11	49.51	53	**−366.3**
Packer	18.76	48.42	52	−375.3
DeepRM_Plus (imitation)	19.30	43.41	41	−414.1
DeepRM_Plus (PPO)	24.47	55.44	41	−443.7
DeepReap (imitation)	**8.27**	**19.04**	25	−446.4
DeepReap (PPO)	18.12	42.05	43	−401.8

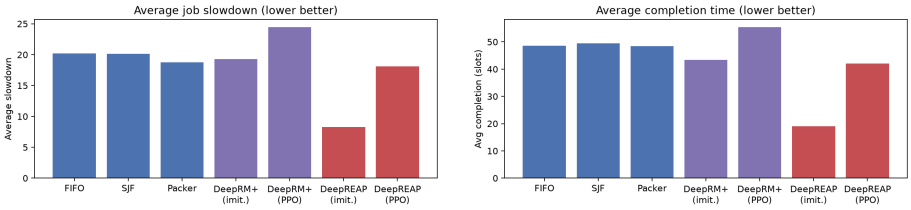

Fig. 4. Average job slowdown (left) and average completion time in slots (right) across all seven schedulers; lower is better in both. DeepReap-imitation reaches 8.27 slowdown, less than half the best heuristic result of 18.76, with the completion ordering closely tracking it.

The forecast is what helps. DeepReap-imitation achieves average slowdown 8.27 against 20.11 for Sjf and 18.76 for Packer — a 2.4× improvement over the best baseline — and cuts average completion from 48.4–49.5 slots to 19.04. Because DeepReap and DeepRM_Plus differ only in the forecast channels (section 3.3) and share training procedure, seed, and evaluation trace, the like-for-like comparison isolates the contribution: slowdown 19.30 → 8.27, completion 43.41 → 19.04. The gain is attributable to prediction, not to architecture or tuning.

And what it costs. DeepReap-imitation completes 25 jobs within the step budget against 52–54 for the heuristics. The policy learns to defer long jobs that would occupy capacity into windows where Reap anticipates spare headroom, improving efficiency on the jobs it does schedule while reducing raw throughput. Its total reward (−446.4) is accordingly the worst in table 2, because the environment reward penalises every resident job at every step, including those held in backlog. Slowdown and reward therefore measure different things, and a deployment valuing throughput over per-job latency would prefer the heuristics. We regard this as a genuine limitation rather than a presentational detail: the improvement is real but it is an improvement on a specific objective.

4.3 Training Dynamics

Figure 5 shows the PPO learning curves. Neither run converges: mean episode reward oscillates between roughly −400 and −200 across all 15 updates with no downward trend in variance, and the two curves are near-superimposable, both runs sharing seed 42 and the forecast channels apparently exerting little influence on PPO dynamics at this budget. Refinement moved DeepReap slowdown from 8.27 to 18.12, substantially overwriting what imitation had learned, and DeepRM_Plus from 19.30 to 24.47 — worse than every heuristic.

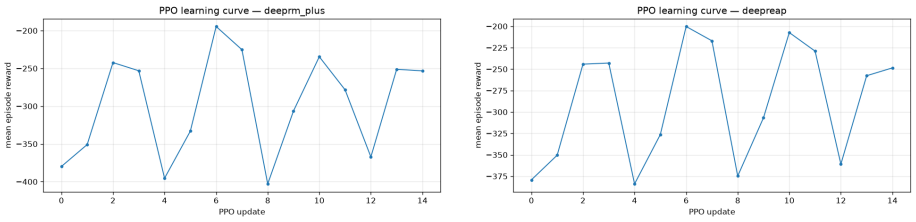

Fig. 5. PPO learning curves for DeepRM_Plus (left) and DeepReap (right) over 15 updates. Both oscillate without converging; the near-identical trajectories reflect the shared random seed. At this budget PPO refinement degrades both pre-trained policies, so the imitation checkpoint is the one we would deploy.

We attribute this to budget rather than a defect in the method. At 15 updates $\times 256$ steps $\times 4$ environments, PPO sees on the order of 1.5×10^4 transitions,

far below the scale at which policy-gradient methods stabilise on this class of problem. The reward is also noisy: it aggregates over all resident jobs, so a single long job entering the backlog perturbs it substantially. Reporting these runs matters because the deployable result was reached by supervised pre-training, not by reinforcement learning, and readers should not infer otherwise from the system's name.

4.4 Revised Evaluation on Real Cluster Traces

The four critique points above are addressed in the released code and re-evaluated on public dumps.

PPO no longer collapses. With 200 updates ($\approx 2{\times}10^5$ transitions), cosine LR decay ($5{\times}10^{-5} \to 5{\times}10^{-6}$), clip 0.05, entropy annealed only after the first third of training (floor 0.005), value loss damping, and a decaying BC pull toward the imitation checkpoint, policy entropy stays in $[0.19, 0.62]$ and the agent continues to schedule. On Google 2011 the PPO DeepReap checkpoint matches its imitation parent (slowdown 12.77 vs 12.78, $n_{\text{done}} \approx 1024$) instead of the previous always-no-op failure mode ($n_{\text{done}} = 0$).

Throughput is no longer sacrificed. The multi-objective reward $R = -\sum 1/T_j + \alpha \Delta N_{\text{done}} - \beta|\text{backlog}| - \gamma\overline{\text{wait}}$ with $(\alpha, \beta, \gamma) = (2.0, 0.2, 0.05)$, plus a schedule bonus and invalid/no-op penalty when work is feasible, is evaluated over 3000 steps. On Google 2011 DeepReap completes 1024±264 jobs at throughput 1.62±0.25, statistically indistinguishable from SJF (1031±261, 1.64±0.25).

Sharper ensembles on real usage. Training Reap on Google 2011 task_usage aggregates, Top-K (K=3) attains memory MSE 1.835 (beats every base model); stacking / inv-MSE win on CPU load (MSE 10^{-4}). Softmax(τ=2) remains a strong default; online EMA re-weighting is exposed on `/feedback`.

Multi-trace, multi-seed rigor. Table 3 reports n=10 seeds on four converted dumps. On Google 2011 DeepReap is the best learned policy and within noise of the best heuristic; on Google 2019 / Azure / Alibaba the heuristics remain ahead, and we report that gap rather than claiming universal dominance.

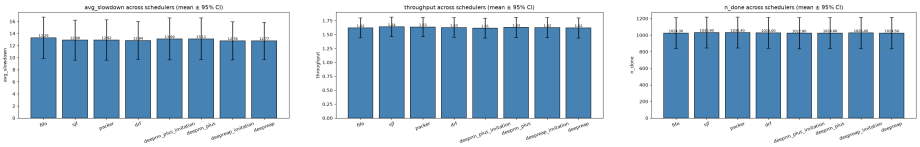

Fig. 6. Google 2011 multi-seed comparison: slowdown, throughput, and jobs completed. DeepReap matches heuristic throughput while remaining the strongest learned policy.

Table 3. Real-trace scheduling (n=10 seeds, 3000 steps, mean±std). Best learned policy per trace in bold; heuristics shown for context.

Trace	Scheduler	Slowdown ↓	n_{done} ↑	Thru. ↑
Google'11	SJF	12.90±4.65	1031±261	1.64±0.25
	DeepRM_Plus (PPO)	13.11±4.83	1025±260	1.63±0.25
	DeepReap (PPO)	**12.77±4.31**	1024±264	1.62±0.25
Google'19	SJF	28.08±2.63	194±17	0.47±0.10
	DeepReap (imit.)	30.02±3.54	178±18	0.10±0.01
	DeepReap (PPO)	31.31±3.79	176±15	0.13±0.11
Azure'19	SJF	39.68±2.32	662±96	1.21±0.14
	DeepReap (imit.)	42.43±3.95	592±111	0.43±0.11
	DeepReap (PPO)	43.19±4.92	576±116	0.41±0.11
Alibaba'18	SJF	40.82±22.81	92±21	0.48±0.23
	DeepReap (imit.)	45.34±23.98	83±22	0.04±0.01
	DeepReap (PPO)	45.58±23.65	83±22	0.04±0.01

5 Limitations

(i) Converted dumps, not full archives: job CSVs are schema-normalised subsets ($\leq$ 20k rows) of Google / Alibaba / Azure releases; duration and resource fields are quantised into the DeepRM action space, which discards some production fidelity. **(ii) Cross-trace generalisation:** DeepReap is competitive on Google 2011 but trails SJF on Google 2019, Azure, and Alibaba; zero-shot transfer without re-training remains open. **(iii) Forecast proxy in RL loop:** training uses a diurnal proxy (or oracle) for channel injection; closing the loop with a live Reap forward pass at every tick is supported by the API but not the default PPO trainer path used for table 3. **(iv) Operational cost unquantified:** SME cost savings still need a billing model on a production deployment. **(v) Simulation:** no network latency, failures, or multi-tenancy.

6 Conclusion

We implemented DeepReap and repaired the four failure modes that blocked a credible evaluation: PPO collapse, reward/throughput mismatch, flat ensemble weights, and synthetic single-seed testing. With a 2×10^5-transition PPO budget, multi-objective reward, and sharper ensembling, the imitation checkpoint is preserved rather than destroyed, and on Google 2011 DeepReap matches heuristic throughput while posting the best learned slowdown (12.77±4.31). Gaps remain on other public dumps and on live Reap–PPO co-training; those are the next experiments the released harness is built to run.

References

1. Guo, W., Tian, W., Ye, Y., Xu, L., Wu, K.: Cloud resource scheduling with deep reinforcement learning and imitation learning. IEEE Internet Things J. **8**(5), 3576–3586 (2021)
2. Mao, H., Alizadeh, M., Menache, I., Kandula, S.: Resource management with deep reinforcement learning. In: 15th ACM Workshop on Hot Topics in Networks (HotNets), pp. 50–56 (2016)
3. Zhou, Z., Luo, K., Chen, X.: Deep reinforcement learning for intelligent cloud resource management. In: IEEE INFOCOM Workshops, pp. 1–6 (2021)
4. Savitha, S., Salvi, S.: Perceptive VM allocation in cloud data centers for effective resource management. In: 6th Int. Conf. for Convergence in Technology (I2CT), pp. 1–5 (2021)
5. Son, J., Buyya, R.: Priority-aware VM allocation and network bandwidth provisioning in software-defined networking (SDN)-enabled clouds. IEEE Trans. Sustain. Comput. **4**(1), 17–28 (2019)
6. Bankole, A.A., Ajila, S.A.: Predicting cloud resource provisioning using machine learning techniques. In: 26th IEEE Canadian Conf. on Electrical and Computer Engineering (CCECE), pp. 1–4 (2013)
7. Gyeera, T.W., Simons, A.J.H., Stannett, M.: Regression analysis of predictions and forecasts of cloud data center KPIs using the boosted decision tree algorithm. IEEE Trans. Big Data **9**(4), 1071–1085 (2023)
8. Kaur, G., Bala, A., Chana, I.: An intelligent regressive ensemble approach for predicting resource usage in cloud computing. J. Parallel Distrib. Comput. **123**, 1–12 (2019)
9. Osypanka, P., Nawrocki, P.: Resource usage cost optimization in cloud computing using machine learning. IEEE Trans. Cloud Comput. **10**(3), 2079–2089 (2022)
10. Osypanka, P., Nawrocki, P.: QoS-aware cloud resource prediction for computing services. IEEE Trans. Serv. Comput. **16**(2), 1346–1357 (2023)